

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ

**Тема: Разработка визуализатора алгоритма Прима
НА ЯЗЫКЕ KOTLIN**

Студент гр. 4343

М.М. Зеров

Студент гр. 4343

М.Д. Геращенко

Студент гр. 4343

В.В. Семенчик

Руководитель

Р.П. Шестопалов

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: М.М. Зеров Группа: 4343

Студент: М.Д. Геращенко Группа: 4343

Студент: В.В. Семенчик Группа: 4343

Тема практики: Разработка визуализатора алгоритма Прима на языке Kotlin

Задание на практику:

Командная итеративная разработка визуализатора алгоритма на Kotlin с графическим интерфейсом.

Алгоритм: Прима.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 17.07.2026

Дата защиты отчета: 17.07.2026

Студент

М.М. Зеров

Студент

М.Д. Геращенко

Студент

В.В. Семенчик

Руководитель

Р.П. Шестопалов

АННОТАЦИЯ

Разработана программа для визуализации алгоритма Прима.. Программа разработана на языке программирования Kotlin с использованием библиотеки JavaFX. В отчёте описаны этапы разработки от создания прототипа до финальной версии. Реализованы функции загрузки и сохранения графов, пошаговая визуализация алгоритма с возможностью автоматического воспроизведения и регулировкой скорости. В программе реализован интерактивный графический интерфейс, позволяющий создавать, редактировать и визуализировать графы, а также отслеживать каждый шаг работы алгоритма Прима.

SUMMARY

A program for visualizing Prim's algorithm has been developed using the Kotlin programming language and the JavaFX library. The report outlines the development stages, ranging from the initial prototype to the final version. Features include graph loading and saving, as well as a step-by-step visualization of the algorithm with automatic playback and adjustable speed. The program incorporates an interactive graphical user interface that allows users to create, edit, and visualize graphs, while also tracking each step of Prim's algorithm as it executes.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ (заголовок второго уровня)5

6

6

6

1.3 Третий раздел первой главы (заголовок третьего уровня)6

7

7

7

Error! Bookmark not defined.

12

13

14

15

Error! Bookmark not defined.

ВВЕДЕНИЕ

В современном мире задачи оптимизации сетей играют важную роль: от проектирования компьютерных сетей до транспортной логистики. Одной из ключевых задач теории графов является поиск минимального остовного дерева. Алгоритм Прима — классический алгоритм для решения этой задачи. Визуализация его работы полезна для образовательных целей учебной практики.

Цель работы — разработать программу для визуализации алгоритма Прима на взвешенных неориентированных графах.

.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Теория графов — раздел дискретной математики, изучающий структуры, состоящие из вершин и рёбер. Графы применяются для моделирования компьютерных сетей, транспортных маршрутов, социальных связей и других систем.

Взвешенные графы содержат числовые значения (веса) у рёбер. Задача поиска минимального остовного дерева (MST) заключается в нахождении подграфа, соединяющего все вершины с минимальной суммой весов рёбер.

Алгоритм Прима — жадный алгоритм, последовательно добавляющий минимальные рёбра. Широко применяется при проектировании сетей и кластеризации данных.

1.2 Задачи практики

Задачи практики:

1. Изучить язык Kotlin[1] и библиотеку JavaFX[2].
2. Разработать программу с интерактивным созданием графов.
3. Реализовать алгоритм Прима с пошаговой визуализацией.
4. Реализовать загрузку и сохранение графов в файлы.
5. Добавить плеер для автоматического воспроизведения шагов.
6. Обработать возможные ошибки.
7. Протестировать программу.

2 РЕЗУЛЬТАТЫ РАБОТЫ

Данная глава должна содержать описание процесса решения и результаты по каждой задаче практики из предыдущего раздела. По приведённому тексту читатель должен понять, *что* и *как* было сделано для выполнения поставленных задач.

Повествование ведется обезличенно.

2.1. Вводное задание:

На начальном этапе практики был изучен язык программирования Kotlin и библиотека JavaFX. В ходе самостоятельной подготовки были освоены темы: работа с контейнерами и менеджерами компоновки, обработка событий мыши и клавиатуры, механизмы перерисовки компонентов, использование Canvas и GraphicsContext для отрисовки графов.

2.2. Составление спецификации программы и плана разработки

2.2.1 Исходные требования к программе

2.2.1.1 Требования к вводу исходных данных:

Входные данные могут быть заданы пользователем интерактивно с помощью мыши на холсте или загружены из текстового файла. Формат файла должен содержать координаты вершин (pos V1 100.0 150.0) и список рёбер с весами (V1 V2 5.0). Поскольку алгоритм работает с неориентированным графом, порядок указания вершин в ребре не имеет значения .

2.2.1.2 Требования к визуализации:

Интерфейс разделён на три области: центральная часть — холст с возможностью прокрутки (ScrollPane) для отображения графа; правая часть — информационная панель с отображением шага, веса, сообщения и списка рёбер в осто́ве; нижняя часть — панель управления, содержащая кнопки для выполнения шагов вперёд и назад, воспроизведения, остановки и регулировки скорости. Вершины отображаются в виде кругов с цветовой индикацией состояния: зелёный — в осто́ве, светло-голубой — обычная, красный контур — выделенная. Рёбра отображаются в виде отрезков с

цветовой индикацией: зелёные — в осто́ве, оранжевые пунктирные — кандидаты. Холст автоматически расширяется при перетаскивании вершин за его границы.

2.2.1.3 Требования к функционалу приложения:

Программа позволяет: создавать и редактировать взвешенный неориентированный граф; пошагово выполнять алгоритм Прима с визуализацией изменений состояний вершин и рёбер; возвращаться на предыдущий шаг; сбрасывать выполнение в начальное состояние; автоматически воспроизводить алгоритм с заданной задержкой; выявлять несвязные графы и выдавать соответствующее сообщение; сохранять и загружать граф в текстовый файл; перемещаться по холсту с помощью полос прокрутки; автоматически расширять холст при перетаскивании вершин за пределы видимой области. Все функции доступны через графический интерфейс.

2.2.2 План разработки и распределение ролей в бригаде

Этап 1, прототип (1-я неделя, 01.07.2026 – 05.07.2026). Реализация базовых классов модели (Vertex, Edge), разработка компонента Canvas для отрисовки графа, реализация обработки событий мыши для добавления вершин и рёбер.

Этап 2, альфа (2-я неделя, 06.07.2026 – 10.07.2026). Реализация алгоритма Прима (PrimEngine) с генерацией пошаговых снимков. Внедрение пошагового режима с цветовой индикацией. Разработка модуля загрузки графа из файла (GraphParser). Создание полноценного пользовательского интерфейса: панель управления (кнопки Play/Pause/Stop, шаг вперёд/назад), информационная панель, слайдер скорости. Реализация сохранения графа и истории шагов (GraphExporter).

Этап 3, бета и релиз (3-я неделя, 11.07.2026 – 15.07.2026). Добавление ScrollPane для прокрутки холста и механизма автоматического расширения холста. Проведение тестирования, устранение дефектов, доработка интерфейса..

Этап 4, отчет (4-я неделя, 16.07.2026 – 18.07.2026): Оформление отчёта.

Распределение ролей:

Разработчик 1 (UI/интеграция) — Зеров Максим. Реализация панелей инструментов и управления, связывание интерфейса с алгоритмами, обработка событий кнопок, вывод результатов и сообщений об ошибках.

Разработчик 2 (модель и визуализация) — Семенчик Влад. Реализация главного окна, отрисовка графа на Canvas, обработка событий мыши на холсте, загрузка и сохранение графа в текстовый файл, вывод результатов и сообщений об ошибках, реализация ScrollPane и авто-расширения холста

Разработчик 3 (алгоритмы) — Геращенко Мария. Реализация классов Vertex, Edge, AlgorithmSnapshot. Реализация алгоритма Прима (PrimEngine), логика пошагового выполнения и отката, управление состояниями вершин, обнаружение несвязных графов.

2.3.1. Структуры данных

Класс Vertex описывает вершину графа и содержит имя, координаты на холсте, радиус и цвет.

Класс Edge описывает ребро и содержит ссылки на начальную и конечную вершины, а также вес ребра.

Класс AlgorithmSnapshot описывает снимок состояния алгоритма и содержит номер шага, множества вершин и рёбер в остова, множество рёбер-кандидатов, лог-сообщение и общий вес построенного дерева.

Класс PrimEngine реализует алгоритм Прима и содержит метод calculatePrim(), возвращающий список снимков состояния.

Все классы моделей реализованы как data class языка Kotlin, что обеспечивает автоматическую генерацию методов equals(), hashCode() и toString().

2.3.2. Основные методы

Класс PrimEngine:

calculatePrim(vertices: List<Vertex>, matrix: Array<DoubleArray>, startVertex: Vertex): List<AlgorithmSnapshot> — основной метод,

реализующий алгоритм Прима. На каждом шаге находит рёбра-кандидаты, выбирает минимальное ребро, добавляет его в остов и сохраняет снимок состояния.

Класс `GraphParser`:

`parseFile(filePath: String): ParsedGraph` — выполняет построчный разбор файла, извлекая координаты вершин, рёбра и историю шагов алгоритма. Поддерживает валидацию входных данных с выводом информативных сообщений об ошибках.

Класс `GraphExporter`:

`saveGraphToFile(filePath: String, vertices: List<Vertex>, matrix: Array<DoubleArray>, mstSteps: List<AlgorithmSnapshot>?)` — сохраняет состояние графа и историю шагов в текстовый файл.

Класс `App` (графический интерфейс):

`drawAll(GraphicsContext)` — основная отрисовка графа: рёбра с цветовой индикацией (зелёные — в остове, оранжевые пунктирные — кандидаты), вершины с цветовой заливкой (зелёные — в остове, светло-голубые — обычные, красный контур — выделенная). Использует методы `GraphicsContext` для рисования фигур, текста и управления стилями линий.

Обработчики событий мыши — создание вершин по щелчку (ЛКМ), создание рёбер между выбранными вершинами (СКМ), удаление вершин и рёбер (ПКМ), перемещение вершин перетаскиванием.

`stepForward()` / `stepBackward()` — пошаговое выполнение алгоритма с возможностью возврата на предыдущий шаг. При выполнении шага обновляется текущий индекс снимка и перерисовывается холст.

`play()` / `pause()` / `reset()` — управление автоматическим воспроизведением с регулируемой скоростью. Используется `Timeline` для периодического вызова `stepForward()`.

`saveGraphToFile()` / `loadGraphFromFile()` — сохранение и загрузка графа с использованием `FileChooser`. При загрузке выполняется автоматическая подгонка размеров холста под загруженный граф.

`updateCanvasSize()` — вычисляет минимальные размеры холста, необходимые для отображения всех вершин с отступами. Автоматически центрирует вершины на холсте.

`distanceToPoint()` — вычисляет расстояние от точки до отрезка, используется для определения клика по ребру при удалении.

2.3.3. Тестирование

Тестирование разработанного приложения выполнялось вручную на всех этапах разработки. Проверке подвергались как отдельные компоненты (расчёт весов рёбер, корректность пошагового выполнения алгоритма), так и работа пользовательского интерфейса в комплексных сценариях: интерактивное построение и модификация графа, импорт данных из файлов, перемещение по шагам алгоритма вперёд и назад, обработка несвязных графов, а также функционирование прокрутки и автоматического расширения рабочей области холста.

Особое внимание уделялось корректности поведения алгоритма на графах разных типов — как связных, так и несвязных, с различным количеством вершин, включая графы с изолированными вершинами. Результаты всех тестов подтвердили корректность разработанного решения: программа успешно строит минимальное остовное дерево для связных графов, правильно вычисляет его суммарный вес, а при обнаружении несвязности информирует пользователя соответствующим сообщением. Интерфейс остаётся отзывчивым при всех операциях, сбоях ввода-вывода и непредвиденных ситуаций в ходе тестирования зафиксировано не было.

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

Графический интерфейс: библиотека JavaFX (классы Application, Stage, Scene, Canvas, GraphicsContext, Button, Slider, VBox, HBox, BorderPane, Alert, TextInputDialog, ScrollPane, FileChooser). Для отрисовки графа на холсте использовались GraphicsContext с методами fillOval, strokeLine, fillText, а также механизмы setLineDashes для создания пунктирных линий при отображении рёбер-кандидатов. Обработка событий мыши реализована через setOnMousePressed, setOnMouseDragged, setOnMouseReleased. Для создания прокручиваемой области использован ScrollPane, что позволяет работать с графами, размер которых превышает видимую область окна. Механизм автоматического расширения холста реализован путём динамического изменения размеров Canvas при перетаскивании вершин за его границы.

Работа с файлами: для загрузки и сохранения графов в текстовых форматах использованы java.io.File с ручным парсингом и валидацией входных данных. Выбор файлов организован через диалоговое окно FileChooser с фильтром расширений (.txt). Реализована поддержка формата, содержащего координаты вершин, список рёбер с весами и опционально историю шагов алгоритма.

Структура проекта: модульная организация исходного кода с разделением на пакеты org.example.newpl.demo (главный класс и интерфейс) и org.example.newpl.demo.backend (модели данных, алгоритм, парсер и экспортёр). Для управления версиями использовалась система Git.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

По результатам работы учебная практика оценена на Отлично (рис. 1–3).

(нужно будет добавить 3 отзыва)

ЗАКЛЮЧЕНИЕ

В ходе учебной практики была выполнена командная разработка программного продукта — визуализатора алгоритма Прима на языке Kotlin с использованием библиотеки JavaFX. Полностью решены все поставленные задачи: изучены основы Kotlin и JavaFX, разработана архитектура приложения с разделением на модель данных, компоненты визуализации и алгоритмическое ядро, реализован алгоритм Прима с поддержкой пошагового выполнения, отката и автоматического воспроизведения с регулируемой скоростью, создан интерактивный редактор графа с масштабируемым холстом, поддержкой прокрутки и автоматическим расширением при перетаскивании вершин, реализованы механизмы загрузки и сохранения графа в текстовом формате, проведено ручное тестирование на графах различных типов (связные, несвязные, с изолированными вершинами) — ошибок и непредвиденного поведения не выявлено.

Разработанное приложение полностью соответствует утверждённой спецификации и может использоваться в учебном процессе для наглядной демонстрации работы алгоритма Прима. Полученные навыки программирования на языке Kotlin, работы с графической библиотекой JavaFX и командной разработки с использованием системы контроля версий будут полезны в дальнейшей профессиональной деятельности.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Введение в Kotlin JVM. URL: <https://stepik.org/course/5448/syllabus> . [1]
2. JavaFX Documentation. URL: <https://openjfx.io/>. [2]